

Exercice 01 — Mettre l'API inf83-api sous Git avec une v1.0.0 et un flux de branches

Projet fil rouge : INF83 – Déploiement continu DevOps - Projet pratique

C'est le **premier exercice** du projet fil rouge ! Vous allez poser les fondations du projet. Projet progressif construit au fil des exercices.

Dans cet exercice : Vous récupérez l'API Node.js/Express + PostgreSQL fournie (routes /health, /ready, /items) avec ses tests Jest. Avant de parler culture DevOps, conteneurs ou CI/CD dans les chapitres suivants, on met le projet sous contrôle de version : dépôt GitHub, .gitignore propre, un flux feat/ -> main, un CHANGELOG et une première release taguée v1.0.0. C'est la base sur laquelle toute la chaîne GitHub -> Actions -> Docker Hub -> VPS reposera.

Avant de commencer

L'**Exemple 01** ([exemples/exemple-01/exemple-01.md](#)) vous a montré, sur une mini lib neutre « calc », **comment** versionner un projet : git init, .gitignore, branche feature mergée, CHANGELOG.md et tags SemVer (v1.0.0 puis v1.0.1). Ici, vous appliquez exactement cette discipline au projet **INF83 – Déploiement continu DevOps - Projet pratique**. Assurez-vous d'avoir lu et suivi l'exemple avant de commencer.

Objectif

Transformer le dossier de l'API fourni en un dépôt Git versionné et publié sur GitHub, avec une première version officielle v1.0.0, un historique passant par une branche de fonctionnalité, un CHANGELOG, et une Issue décrivant un bug reproductible. (La protection de branche et la revue par PR viendront à l'Exercice 02 ; ici on pose les fondations.)

Prérequis

- Connaissances en développement d'applications (niveau CDA)
- Bases de la ligne de commande
- Notions de développement web et d'API
- Comptes GitHub, Docker Hub et accès à un VPS créés avant le J1

Scénario

Vous êtes intégré dans une équipe qui vous confie l'API inf83-api. Le code marche en local (`npm test` est vert) mais il n'est ni versionné ni publié. Votre mission : le mettre sous Git proprement, sortir une v1.0.0, et signaler un bug que vous avez réussi à reproduire pour que l'équipe puisse le traiter.

Étapes à réaliser

Étape 1 — Récupérer l'API et vérifier qu'elle tourne

Partir d'une base saine : le code fourni doit passer les tests avant d'être versionné.

Ce que vous devez faire :

- Récupérer le dossier `inf83-api/` fourni (API Express + PostgreSQL, routes `/health`, `/ready`, `/items`).
- Installer les dépendances avec `npm install`.
- Lancer `npm test` et vérifier que les tests Jest sont verts.

Résultat attendu : `npm test` se termine sans échec. Vous savez que la base est saine avant de la versionner.

Erreur fréquente : Versionner un projet sans vérifier les tests, puis taguer une `v1.0.0` cassée. On ne tague que du code sain.

Étape 2 — Créer le dépôt GitHub vide et le `.gitignore`

Initialiser le versionning localement en ignorant ce qui ne doit jamais entrer dans l'historique.

Ce que vous devez faire :

- Créer sur GitHub un dépôt **vide** nommé `inf83-api` (sans README, sans `.gitignore`, pour éviter un historique divergent au premier push).
- Dans le dossier local, initialiser git (`git init`) si ce n'est pas déjà fait.
- Créer (ou compléter) un `.gitignore` contenant au minimum `node_modules` et `.env`.

Résultat attendu : Le dépôt distant existe et est vide ; le `.gitignore` local ignore `node_modules` et `.env`.

Erreur fréquente : Cocher 'Add a README' à la création du dépôt puis se retrouver avec un historique divergent au premier push. Créez le dépôt vide.

Étape 3 — Premier commit et push sur main

Enregistrer l'état initial du code et le publier sur la branche main.

Ce que vous devez faire :

- `git add .` puis vérifier avec `git status` que `node_modules` et `.env` ne sont PAS dans les fichiers à commiter.
- Faire un premier commit clair (par exemple `chore: import initial de l'API inf83`).
- Relier le dépôt local au distant (`git remote add origin ...`) et pousser sur `main`.

Résultat attendu : Le code de l'API est visible sur GitHub, branche main, avec au moins un commit. `node_modules` n'apparaît pas.

Piste : Si `git status` montre `node_modules`, c'est que le `.gitignore` n'est pas pris en compte (mauvais emplacement ou `node_modules` déjà indexé : `git rm -r --cached node_modules`).

Étape 4 — Créer une branche de fonctionnalité pour une petite évolution

Mettre en place le flux `feat/` -> `main` avec un changement minuscule.

Ce que vous devez faire :

- Créer une branche `feat/` depuis `main` à jour (par exemple `feat/route-version`).

- Faire une petite evolution : par exemple exposer la version (ajouter une route GET /version qui renvoie la version de package.json) OU enrichir le README. Gardez ca minuscule.
- Commiter sur la branche.

Résultat attendu : Une branche `feat/...` existe avec un commit contenant votre petite evolution.

Piste : L'objectif ici est le **flux** (branche -> merge), pas la fonctionnalite. Un petit changement suffit largement.

Étape 5 — Merger la branche dans main

Integrer la fonctionnalite dans main pour garder un historique lisible.

Ce que vous devez faire :

- Revenir sur `main` (`git checkout main`).
- Merger la branche (`git merge feat/route-version`).
- Supprimer la branche de feature une fois mergee (`git branch -d feat/route-version`) et pousser main.

Résultat attendu : main contient votre evolution, l'historique montre l'integration de la branche, la branche de feature est supprimee.

Erreur fréquente : Coder directement sur main « parce que c'est plus rapide ». On perd l'interet du flux par branche et la tracabilite du changement.

Étape 6 — Ecrire le CHANGELOG.md

Documenter la premiere version de maniere lisible par un humain.

Ce que vous devez faire :

- Creer un `CHANGELOG.md` a la racine.
- Y ajouter une section `## [1.0.0]` avec les sous-sections **Added / Changed / Fixed**.
- Decrire ce que contient cette premiere version (routes /health, /ready, /items, et votre petite evolution).

Résultat attendu : Un `CHANGELOG.md` present sur main, avec une entree 1.0.0 lisible.

Piste : Inspirez-vous de Keep a Changelog. Meme si Changed/Fixed sont vides pour une v1.0.0, gardez les sous-titres pour le format.

Étape 7 — Poser et pousser le tag v1.0.0

Materialiser la premiere release officielle avec un tag SemVer.

Ce que vous devez faire :

- Verifier que main est a jour et que `npm test` passe.
- Poser un tag annote : `git tag -a v1.0.0 -m "Premiere version stable de inf83-api"`.
- Pousser le tag : `git push origin v1.0.0`.

Résultat attendu : Le tag v1.0.0 apparait sur GitHub (onglet Tags / Releases) et pointe sur le commit de release.

Erreur fréquente : Croire que `git push` envoie aussi les tags. Il faut `git push origin v1.0.0` (ou `git push --tags`), sinon le tag reste local.

Étape 8 — Ouvrir une Issue GitHub pour un bug reproductible

Pratiquer la gestion de probleme : signaler un bug de maniere exploitable par l'equipe.

Ce que vous devez faire :

- Reperer (ou imaginer) un bug **reproductible** : par exemple, appeler GET /ready quand la base PostgreSQL n'est pas joignable.
- Ouvrir une **Issue** sur GitHub avec : un titre clair, les etapes pour reproduire, le resultat observe vs attendu, et un debut de **diagnostic de 1er niveau** (logs consultes, route testee, hypothese).
- Indiquer si le probleme doit etre **escalade** (ex. bug bloquant -> assigner / labelliser).

Résultat attendu : Une Issue ouverte, structuree, qu'un collegue peut reproduire sans vous poser de question.

Piste : Un bon rapport de bug = quelqu'un d'autre reproduit le probleme en suivant vos etapes. Si ce n'est pas reproductible, ce n'est pas (encore) exploitable.

Vérification

Quand vous avez terminé, vérifiez que :

- ☐ Le depot GitHub `inf83-api` contient le code de l'API sur main.
- ☐ `node_modules` et `.env` ne sont PAS versionnes (`.gitignore` en place).
- ☐ L'historique montre une branche `feat/` mergee dans main.
- ☐ Un `CHANGELOG.md` avec une entree 1.0.0 (Added/Changed/Fixed) est present.
- ☐ Le tag `v1.0.0` est visible sur GitHub.
- ☐ Une Issue decrit un bug reproductible avec un diagnostic de 1er niveau.

En pratique

Sur un projet reel, le numero de version n'est pas decoratif : c'est lui qui dira plus tard quelle image Docker deployer et sur quoi faire un rollback. Prenez maintenant l'habitude de ne taguer que du code teste et merge sur main. Et soignez vos Issues : un bug bien decrit (reproductible, avec diagnostic) se corrige vite ; un « ca marche pas » fait perdre des heures a toute l'equipe.

Bonus (Facultatif mais Recommandé)

Creer une **Release GitHub** a partir du tag `v1.0.0` (onglet Releases -> Draft a new release) en y collant les notes du `CHANGELOG`. Bonus supplementaire : adopter les Conventional Commits (`feat`:, `fix`:, `chore`:) pour pouvoir generer un changelog automatiquement plus tard.

Livable attendu

URL du depot GitHub [inf83-api](#) montrant : le code de l'API sur main, le tag v1.0.0 dans les tags/releases, le CHANGELOG.md a la racine, l'historique avec la branche feat/ mergee, et le lien vers l'Issue du bug reproductible.

Une fois terminé, consultez la **correction** dans [corrections-exercices/correction-exercice-01/correction-exercice-01.md](#) pour comparer votre travail.